



Optimizing Service Placement in Fog Environments with Deep Reinforcement Learning

Wei Liu^{1,2(✉)}, Qin Xie², Tariq Ali Arain², and Usama Ali²

¹ Key Laboratory of Water Big Data Technology of Ministry of Water Resources, Hohai University, Nanjing, China

liu_wei912@163.com

² College of Computer and Information, Hohai University, Nanjing, China

Abstract. Fog computing serves as an intermediary between cloud computing and end users, enhancing latency and Quality of Service (QoS) by deploying resources at the network's edge. The challenge of efficient service placement in fog environments arises from resource heterogeneity, network instability, and task dependencies. This paper introduces a hierarchical service placement framework based on deep reinforcement learning (HA3C). The framework employs a hierarchical decision mechanism to divide the service placement problem (SPP) into two subproblems: community selection and node selection. Our approach incorporates a dynamic multi-objective reward function, introducing a Load Balancer index while optimizing service delay and resource utilization. Extensive simulations on the iFogSim platform reveal that our method significantly outperforms basic algorithms in fog node placement rate while maintaining moderate to high resource utilization. In particular, HA3C exhibits enhanced robustness and scalability in large-scale tasks and complex network topologies. The experimental results confirm the efficacy of our method in optimizing service placement decisions in fog environments.

Keywords: Fog computing · Service Placement · Resource heterogeneity · Load balancing · Deep Reinforcement Learning

1 Introduction

With the explosive growth of Internet of Things (IoT) devices and applications, a massive number of computation-intensive tasks have emerged that impose strict requirements on both latency and energy consumption. Due to limitations in local resources, end devices struggle to meet the computational demands of these complex applications. To address this issue, fog computing (FC) has emerged as an innovative computing paradigm [1]. Fog computing deploys computational nodes between user devices and remote cloud data centers, bringing computational resources closer to the data source, thereby reducing task processing latency and energy consumption [2]. However, efficient service placement in

fog computing remains challenging due to the need to accommodate dynamic changes in fog node resources, such as compute power, memory, and bandwidth, and inter-task dependencies.

The dynamic nature of user requirements and fog node resources renders the service placement problem NP-hard, and some heuristic methods struggle to adapt to real-time adjustments [3,4]. Traditional centralized deep reinforcement learning algorithms, such as DQN and PPO [5,6], encounter significant challenges in complex fog computing environments. These algorithms typically use a single-agent architecture, which suffer from low sampling efficiency and slow training processes, often falling into local optima in high-dimensional state spaces. Furthermore, existing work that applies reinforcement learning to address the SPP in dynamic environments has focused on energy consumption and latency [7,8]. The existing research still face the following issues in the process of generating service placement strategies:

- Reducing latency and energy consumption is crucial in system design, yet the SSP must also prioritize resource balance to ensure long-term stability in fog environments. Load balancing is essential; Without a load balancing strategy, some FNs may become overloaded while others remain underutilized [9]. This imbalance not only leads to resource wastage, but also causes a sharp increase in task response time and energy consumption on overloaded nodes, potentially resulting in system instability. However, current research lacks global awareness of the load state in fog networks, making it difficult to effectively coordinate load distribution in multi-objective optimization, thus failing to maintain system stability under varying workloads.
- The sequential constraints on task execution within a service that the placement of subsequent tasks is contingent upon the placement of preceding ones. The heterogeneity and limited resources of nodes in fog environments further complicate these decisions. Current algorithms have not adequately considered the hierarchical structure of fog computing nodes, neglecting resource and communication disparities across different levels, which leads to suboptimal service placements. Additionally, when handling complex services, failing to address inter-task dependencies increases cross-node communication overhead and execution delays.

This paper proposes a service placement framework based on Deep Reinforcement Learning (DRL) that takes into account the dependencies between multiple tasks within a service. Initially, tasks are clustered based on their dependency strength. Subsequently, FNs are hierarchically partitioned and grouped into communities. The DRL algorithm then selects suitable communities for these task clusters. A spatiotemporal fusion network [10] model is employed to capture the temporal characteristics of system states and the spatial characteristics of system resources. The enhanced Asynchronous Advantage Actor-Critic (A3C) algorithm is employed to facilitate a multilevel decision-making process. The A3C algorithm is selected as the core due to its distributed architecture [11], enabling multiple parallel agents to asynchronously interact with the envi-

ronment and update shared network parameters, ensuring efficient and stable training. The main contributions are as follows:

- We propose a two-level decision-making framework utilizing the A3C algorithm called HA3C. The initial phase involves selecting the optimal device community for service deployment via a reinforcement learning approach. Subsequently, a heuristic method identifies specific nodes, effectively adapting to the hierarchical characteristics of fog computing. The first phase's multi-objective reward function integrates system node load balancing and incorporates the Jain fairness index. Load considerations are also integrated into the second phase, preparing the system for future high loads.
- We propose a hybrid adaptive placement strategy that leverages task dependency strength to implement a hierarchical decision-making process: “task clustering → node partitioning → device selection.” This approach aggregates fog node resource attributes and geographic characteristics to manage multi-task dependencies within services, ensuring efficient computing resource allocation and optimized communication. Utilizing a spatio-temporal fusion network integrated with the A3C learning architecture, our method effectively captures temporal features and topological information in fog environments, enhancing the model's adaptability to dynamic conditions. Additionally, we apply modern regularization techniques, including LayerNorm and Dropout into the A3C network, improving the model's robustness and generalization in noisy settings.
- This paper evaluates the performance of the proposed algorithm through simulation experiments. By comparing it with various other algorithms, the results demonstrate the effectiveness of the proposed approach. The findings confirm that the proposed algorithm improves the fog node placement rate while balancing resource utilization.

The rest of this paper is organized as follows: Section 2 introduces the related work. Section 3 describes the details of the SPP model in FC. Section 4 describes the proposed service placement method. Section 5 presents the experimental evaluation. Finally Sect. 6 concludes the paper.

2 Related Work

Existing service placement optimization methods can be primarily categorized into mathematical modeling-based approaches and reinforcement learning-based approaches.

2.1 Mathematical Models-Based Method

Early research primarily focused on mathematical modeling and heuristic algorithms to address the service placement problem in fog computing. Souza et al. [12] explored cloud-fog collaboration by modeling service allocation as an ILP and introduced a greedy heuristic algorithm to optimize placement by minimizing service delay and meeting capacity constraints. To handle the multi-objective nature

of the service placement problem, Trabelsi et al. [13] defined the Service Placement Problem (SPP) as a Mixed-Integer Linear Programming (MILP) model and designed a multi-objective dynamic service placement (moDSP) heuristic method. This method dynamically filters and evaluates candidate fog nodes, selecting the optimal placement location. Considering the unique characteristics of the Internet of Things (IoT) scenario, Tavousi et al. [14] proposed a heuristic strategy specifically for IoT service placement under a hierarchical fog-cloud architecture, aiming to provide high QoS services to IoT users while minimizing overall resource wastage. Natesha et al. [15] defined Fog Service Placement Problem (FSPP) as a multi-objective optimization problem, employing an elite strategy-improved genetic algorithm (elite-based GA) to optimize system energy consumption, service time, and deployment cost concurrently. Addressing the incremental nature of service placement in dynamic fog environments, Samani et al. [16] proposed a hierarchical optimization method based on resource partitioning and similarity mapping to enhance placement efficiency by slicing heterogeneous fog resources. Wang et al. [17] conceptualized SPP as a dynamic multi-objective optimization problem and developed a two-stage heuristic algorithm that employs cross-mutation and local search strategies to achieve joint optimal placement of services and data.

2.2 Reinforcement Learning-Based Methods

Reinforcement learning has recently garnered significant attention for its superior performance in complex decision-making tasks and is now being applied to optimize service placement in fog computing environments. Goudarzi et al. [18] proposed a distributed deep reinforcement learning approach based on the IMPALA model, termed X-DDRL, which employs experience sharing and a distributed actor-learner architecture to achieve efficient adaptive online service placement. Simulation results indicate that this method markedly enhances system response times. Considering the dynamic changes in user requests and node states, Sami et al. [19] proposed a demand-driven deep reinforcement learning (DRL) service placement strategy. This strategy leverages a DRL model to forecast future fog node states and service requests, autonomously learning placement strategies to minimize service startup times and optimize system performance by preemptively placing service copies. To minimize the latency of service data access, Mangiaracina et al. [20] implemented an adaptive multi-agent reinforcement learning approach to dynamically optimize data placement. By utilizing near-real-time data replication, migration, and referencing strategies, they collaboratively enhance data availability and locality, thereby accelerating service execution.

While previous efforts have optimized fog computing service placement, existing reinforcement learning approaches predominantly address energy and latency tradeoffs, often neglecting the influence of multi-task dependencies on placement decisions. As service structures grow more complex and the number of tasks and fog nodes increases, the efficiency and performance of current algorithms may decline, challenging their ability to maintain stability in large-scale systems.

3 System Model

In this section, we begin by presenting the system architecture of the intelligent service placement framework in fog computing environment. We then construct a multi-layer topology model for the fog computing network, establishing a hierarchical decision-making mechanism through community-based management of fog nodes according to resource characteristics.

3.1 FC Model

Our system employs a three-tier cloud-fog-edge architecture, as depicted in Fig. 1, to establish a fog computing environment. The cloud layer serves as a high-performance data center, suitable for hosting computation-intensive and complex applications [21]. Positioned between the cloud and end-users, the fog layer $F = (D, FNs)$ offers localized computing and storage services. Edge devices submit service placement requests to agents at the network's edge to achieve higher bandwidth and lower latency. These agents manage the chaining of directed acyclic graphs (DAGs), fog system states, and constraints to determine task placement for each service request. Tasks may be assigned to fog nodes (FNs) or cloud servers (CSs) for execution. Each fog node has limited computational and storage capacity, denoted as $FNs = 1, 2, \dots, N$, where the i th fog node FN_i can be represented as:

$$FN_i = (fr_i, mem_i, core_i, R_i, BW_i) \quad (1)$$

fr_i , mem_i , $core_i$, R_i , and BW_i represent the CPU frequency (MIPS), memory capacity (MB), number of cores, storage size (MB), and bandwidth (Mbps) of FN_i respectively.

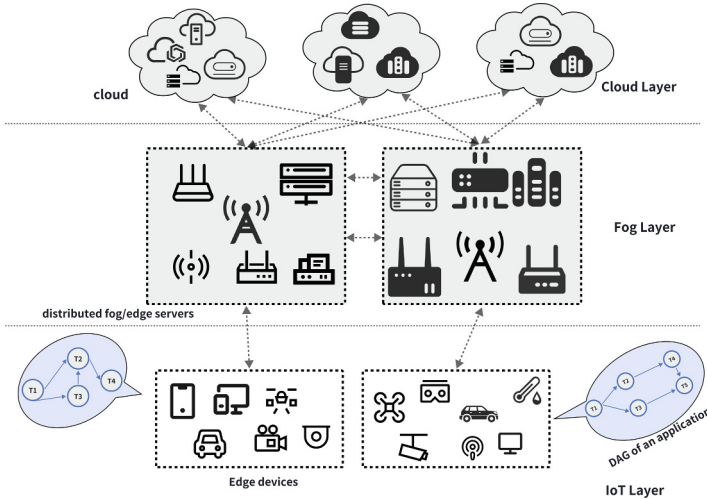


Fig. 1. Overview of our system model.

3.2 Fog Resource Model

In fog environments, there is a large number of nodes with significant heterogeneity. To better handle resource diversity and facilitate resource management, we divide the fog nodes into multiple independent communities, where devices within each community exhibit resource similarities to simplify management.

In Fig. 2 FNs can be represented as a multilayer graph $G = (D, E, L)$, where D is the set of fog devices replicated across all layers. $E = \{E_{ll'} | \forall l, l' \in [0, L]\}$ represents the edges connecting devices across different layers. $E_{ll'} = \{E_{ll'}^j | (d_j, d_j) \in D \times D\}$ denotes connections between devices in different layers of the same device. $L = \{l_1, l_2, l_3, l_4\}$ represents different resource layers: l_1 is the node network topology layer represents device connectivity, l_2 is the processing layer represents CPU frequency and core count, l_3 and l_4 represent the memory and storage layers. The intra-layer edges $e_{ll} = \{e_{jk}^l | (d_j, d_k) \in D \times D, j \neq k\}$ connect devices within the same layer based on similarity. The similarity score $w_{jk}^{(l)} = \frac{1}{1 + \text{dis}_l(d_j, d_k)}$, where $\text{dis}_l(d_j, d_k)$ is the Euclidean distance between resources, is calculated sequentially based on the features of each layer, and is defined as follows:

$$\text{dis}_l(d_j, d_k) = \begin{cases} \sqrt{(R_{j1} - R_{k1})^2 + (R_{j2} - R_{k2})^2}, & l = l_1 \\ R_{jl} - R_{kl}, & l \in \{l_2, l_3, l_4\} \end{cases} \quad (2)$$

R_{j1} and R_{j2} represent the normalized results for the network and processing layers of device d_j . In Fig. 2 the colored ellipses represent different partitioned communities. By using modularity metrics to measure the connection strength between nodes, the multilayer graph of fog nodes is transformed into multilayer partitions:

$$Q = \frac{1}{2W} \cdot \sum_{l \in L} \sum_{l' \in L} \sum_{d_j \in l} \sum_{d_k \in l} \left\{ \left(w_{jk}^{(l)} - \frac{\sigma_j^l \cdot \sigma_k^l}{2W_l} \right) \cdot \alpha_{ll'} + B_{jk} \cdot E_{ll'}^k \right\} \cdot \lambda_{jk} \quad (3)$$

$\sigma_j^{(l)} = \sum_{d_j \in l} w_{jk}^{(l)}$ and $\sigma_k^{(l)} = \sum_{k \in l} w_{jk}^{(l)}$ represent the connection strength of device nodes d_j and d_k with the devices in layer l . B_{jk} is equal to 1 if $j = k$ and 0 otherwise; $W_l = \sum_{d_j \in l} \sum_{d_k \in l} w_{jk}^{(l)}$ is the total sum of intra-layer edge weights for devices within layer l . W is the total sum of intra-layer edge weights across all fog device layers.

The resulting multilayer partitions can then be viewed as nodes, and further compression between layers results in disjoint partitions. Each feature partition groups devices with similar resource characteristics.

3.3 Service Model

Each client submits a service request comprising multiple interdependent tasks, represented as a DAG, $G = (V, E)$. Where $V = \{v_i | 1 \leq i \leq |V|\}$, and $|V| = L$. v_i represents the i th task, $|V|$ indicating the number of tasks. The dependency set



Fig. 2. From Hierarchical to Regional Division of Fog Nodes.

$E = \{e_{i,j} \mid v_i, v_j \in V, i \neq j\}$ represents the data communication between tasks, where each edge $e_{i,j}$ indicates that task v_j cannot execute before task v_i . The task clustering approach quantifies the dependency strength between tasks through the required communication bandwidth, aiming to co-locate communication-intensive tasks to minimize network overhead. The communication strength between tasks is represented as follows: $C_{i,j} = \frac{\min(B_{up,i}, B_{down,j})}{B_{max}}$, $B_{(up,i)}$ is the upstream bandwidth of task v_i , representing the bandwidth required for task v_i to send data to v_j . $B_{down,j}$ is the downstream bandwidth of task v_j , representing the bandwidth required to receive data from task v_j . B_{max} is the maximum bandwidth supported by the system, representing the maximum data transmission rate of the network or device. This approach reduces cross-partition or cross-device data transfers. Task clusters with strong dependencies are prioritized for placement in the same partition, or even on the same device, to reduce inter-partition or inter-device data transmission.

3.4 Service Placement Model

We model the service placement problem in fog environments as a multi-objective optimization problem. The total system cost is expressed as the weighted sum of response time, resource utilization, and load balancing metrics. The objective is to minimize the overall system cost while satisfying the resource constraints of the fog computing system. The specific optimization formula is as follows:

$$P_1 : \min \sum_{s \in S} \sum_{i \in T_s} \sum_{j \in D} x_{ij} \cdot \text{Cost}_{ij}^{T_i} + \lambda \cdot (1 - J) \quad (4)$$

$$\text{s.t.} \quad C_1 : \sum_{j \in D} x_{ij} = 1, \forall i \in T \quad (5)$$

$$C_2 : \sum_{i \in T} x_{ij} \cdot \text{cpu}_i \leq \text{CPU}_j, \forall j \in D \quad (6)$$

$$C_3 : \sum_{i \in T} x_{ij} \cdot \text{mem}_i \leq \text{MEM}_j, \forall j \in D \quad (7)$$

$$C_4 : start_i \geq finish_k, \forall i \in T, \forall k \in dep(i) \quad (8)$$

$$C_5 : \sum_{i,l \in C_m} diff(x_{ij}, x_{lj}) \leq \theta_m \cdot |C_m|, \forall C_m \in CL \quad (9)$$

S denotes the set of all services in the system, and for each service $s \in S$, T_s denotes the set of all tasks contained within service s . C_1 specify that each task is placed on exactly one device. C_2, C_3 specify that the resources allocated to each task do not exceed the capacity of the respective device. C_5 enforces the dependency constraint between tasks, ensuring that each task starts execution only after all its dependent tasks have been completed. C_6 Represents the intra-cluster cohesion constraint, ensuring that related tasks within the same cluster C_m are prioritized for placement in the same partition.

4 Hierarchical Service Placement Algorithm

The SPP has been proven to be NP-hard [22], we propose a reinforcement learning-based service placement algorithm. This algorithm prioritizes load balancing as the core objective while also considering latency and resource utilization. This section will provide a detailed description of the algorithm's design and implementation, detailed in Algorithm 1.

Algorithm 1 Hierarchical A3C Service Placement Algorithm

```

1: Initialize environment (device, topology, partition information)
2: Initialize experience buffer
3: for episode=1,2,...,E do
4:   Reset environment state and agent hidden state
5:   for time slot  $t = \mathcal{T}$  do
6:     Cluster tasks into groups  $\{c_1, c_2, \dots, c_k\}$  based on dependency strength
7:     for each task cluster  $c \in \{c_1, c_2, \dots, c_k\}$  do
8:       Select optimal partition  $p$ 
9:       If  $p$  satisfies resource constraints then
10:        Place task cluster in partition  $p$ 
11:       else
12:        Execute cross-partition optimization or cloud offloading
13:       end if
14:     end for
15:   Advance environment time and update resource states  $s_{t+1} \leftarrow Update(s_t)$ 
16:   Store experience into buffer
17:   Update A3C network parameters  $\theta \leftarrow \theta - \alpha \nabla L(\theta)$ 
18: end for
19: end for
Output: Service placement policy

```

4.1 Partition Selection Based on A3C

We designed an A3C architecture to learn the service placement strategy. This network consists of the following components:

Spatiotemporal Fusion. We employ ConvLSTM (Convolutional LSTM) [23] architecture to effectively capture both spatial and temporal features simultaneously. Unlike traditional approaches that process spatial and temporal information separately, ConvLSTM incorporates convolutional operations directly within LSTM cells, creating a unified structure that preserves spatial correlations across time steps. ConvLSTM maintains the topological structure of input data throughout the computational process. By replacing matrix multiplications with convolution operations, spatial dependencies are preserved in the cell state and hidden state, creating a memory mechanism that is spatially coherent - essential for tracking resource distribution patterns across fog communities. This architecture is particularly effective for fog computing environments where resource distribution patterns evolve dynamically across both space and time dimensions. Our ConvLSTM ingests an input tensor:

$$S_t \in \mathbb{R}^{\text{batch_size} \times \text{time_steps} \times \text{channels} \times \text{height} \times \text{width}} \quad (10)$$

where the channels dimension encodes distinct resource metrics and height and width represent the community topology. By maintaining spatial structure throughout the temporal sequence, the model can continuously track and adapt to evolving resource patterns and inter-community relationships. From an information theoretic perspective, ConvLSTM optimizes the mutual information between consecutive frames of spatial data, effectively modeling the conditional entropy of future resource states given the observed history.

Actor-Critic Architecture. We use the A3C architecture to address the service placement problem in fog computing. This approach benefits from parallel computing to optimize deep neural network parameters asynchronously, enabling efficient learning in complex state spaces. As shown in Fig. 3, the A3C framework includes a central parameter server and multiple parallel agents arranged in a star topology. The central server holds the global policy and value function parameters. Each agent has components for interacting with the environment and updating parameters, performing independent state-action trajectory sampling and gradient computation. Local parameters are periodically synchronized with the global network for distributed learning. The agent interacts with the environment, generating experience trajectories $\tau = (s_t, a_t, r_t, s_{t+1})_{t=0}^T$. The n-step return and Temporal Difference (TD) error are used to estimate the value gain of specific actions relative to average performance, providing a low-variance baseline for strategy gradients. The innovation of A3C lies in its asynchronous update mechanism, where each agent computes the policy and value gradients independently and updates the global network non-blockingly. The Actor-Critic structure uses two networks: the Actor learns the strategy and generates action distributions, while the Critic evaluates states and guides the Actor. Both networks share a feature extraction layer but produce distinct decision signals.

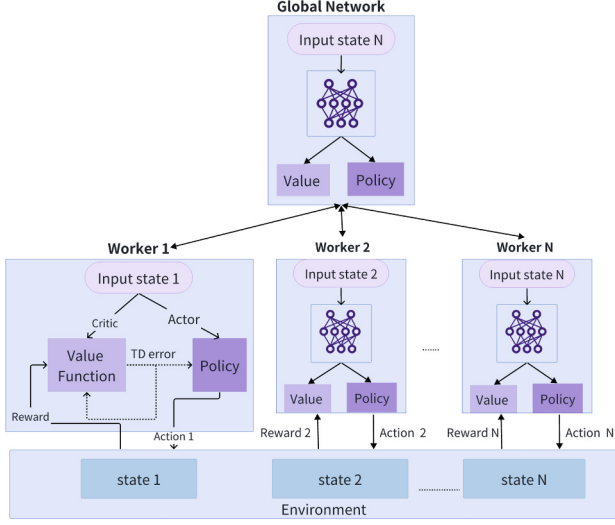


Fig. 3. The architecture of A3C.

4.2 Multi-objective Decision Optimization

The process of solving problems using reinforcement learning is typically modeled as a Markov Decision Process (MDP).

State Space. In our proposed reinforcement learning framework, the design of the state space is crucial for enabling intelligent decision-making. The quality of state representation directly influences the model's comprehension of the environment and decision-making accuracy. We construct a multi-dimensional, multi-level state space that captures the complete environmental information from three perspectives: task characteristics, global system load, and community-specific features, providing the agent with comprehensive decision-making criteria.

Action Space. We propose a novel hierarchical action space structure. This approach significantly reduces decision complexity and enhances learning efficiency. The action space is defined as follows: $A = \{a_t\}$, $a_t = (\text{comm}, \text{node})$.

This two-stage decision mechanism decomposes the originally high-dimensional joint action space into two lower-dimensional subspaces for sequential decision-making, significantly reducing the complexity of the action space from $O(n_{\text{total}})$ to $O(n_{\text{comm}} + n_{\text{node}})$, where n_{total} is the total number of nodes in the system, which is much larger than $n_{\text{comm}} + n_{\text{node}}$. In the first stage, decision-making centers on selecting suitable communities (partitions) based on community-level attributes, such as overall resource availability, load balancing, and topological features, to identify the community that best aligns with

the task’s requirements. In the second phase of decision-making, specific nodes within the identified community are selected. This phase evaluates node-level features, including resource compatibility, current loads, and communication costs, to determine the most suitable nodes for the task. The second-stage heuristic node selection method offers advantages of low computational overhead and fast decision-making, enabling real-time response in resource-constrained fog environments.

Reward. In a distributed fog computing environment, service placement decisions need to balance multiple objectives, including latency, resource utilization, and load balancing. To achieve this complex goal, we propose a comprehensive reward function that coordinates the optimization objectives across different dimensions through a weighting mechanism:

$$R(s_t, a_t) = w_1 \cdot R_{\text{delay}} + w_2 \cdot R_{\text{balance}} + w_3 \cdot R_{\text{comm}} \quad (11)$$

The latency reward term encourages the agent to select placement strategies that minimize task execution delay, directly affecting QoS. By directly quantifying the impact of placement decisions on the system’s load balancing state, the reward function incentivizes the agent to learn load-balancing strategies. The load balancing reward is defined as: $R_{\text{balance}} = J_{\text{after}} - J_{\text{before}}$. Where J_{before} and J_{after} represent the Jain fairness indices before and after placement, respectively. The Jain index is calculated as: $J = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2}$, x_i is the load value of each device and n is the number of devices. This index ranges from $[0, 1]$, with a value closer to 1 indicating a more balanced load distribution.

5 Experimental Evaluation

In this section, we design different experiments to validate the effectiveness of the proposed method. The objectives of the experiments are to investigate the following questions:

- **RQ1:** Is the HA3C algorithm more effective in service placement decisions compared to other reinforcement learning algorithms?
- **RQ2:** How does the performance of the HA3C algorithm change as the number of tasks increases?

5.1 Experimental Setup

The model is trained with a computer with an Intel(R) Core(TM) i7-13500H processor operating at 2.60 GHz, 16.0 GB of RAM, running the Windows 11 operating system, and featuring an NVIDIA GeForce RTX 4090 GPU. The algorithm parameters are configured as shown in Table 1. All experiments were performed with the same parameters, which are considered optimal settings based on our experimental observations.

Table 1. Algorithm Parameters

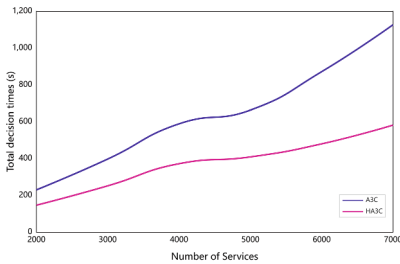
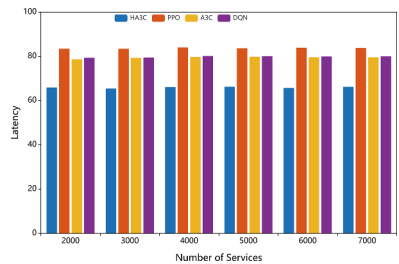
LSTM Parameters	Value	A3C Parameters	Value
Architecture	ConvLSTM	Learning rate	0.0001
Hidden channels	[32,64]	Discount factor	0.99
Number of layer	2	Entropy coeff	0.15
Learning rate	0.0001	Value loss coeff	0.5
Weight decay	0.00001	Update frequency	20
Optimizer	AdamW	Number of workers	3

5.2 Experiment Data

We utilized the iFogSim [24], simulation tool to model the cloud-fog-edge computing environment. Device and network topologies were constructed using iFogSim, with multiple fog computing nodes specified, each assigned resources such as CPU, memory, and storage. The network topology was then established, detailing communication latency, bandwidth, and other network parameters. These configurations form the foundational environment for subsequent service placement.

5.3 Experimental Results

As shown in Fig. 4, the comparison of decision times clearly demonstrates the efficiency gains achieved by the hierarchical decision-making mechanism. As the number of services increases from 1000 to 7000, the decision time growth for HA3C is significantly slower than that of traditional A3C. When the service count reaches 7000, HA3C requires only approximately 800 s for decision-making, while A3C exceeds 1200 s. These results compellingly validate our proposed hierarchical strategy—which decomposes the service placement problem into community selection and node selection subproblems—as an effective means of reducing decision-space complexity, with particularly pronounced advantages in large-scale service scenarios.

**Fig. 4.** Total decision times.**Fig. 5.** Service latency.

To address RQ1, we compared the performance of several classical reinforcement learning algorithms on the service placement problem. Figure 5 presents the service processing latencies achieved by each algorithm. Although latency rises for all methods as service count increases, HA3C consistently attains the lowest delay. At 6000 services, HA3C records 62 ms compared with 78ms for A3C, 82ms for PPO, and 79ms for DQN, indicating that HA3C’s allocation strategy more effectively minimizes processing delays. Figure 6 reports the fog node placement rates. HA3C sustains approximately 80% placement under every workload, outperforming A3C (74%), PPO (71%), and DQN (68%). These results demonstrate that HA3C sharply reduces dependence on cloud resources, thereby lowering network latency and bandwidth consumption.

Figure 7 shows resource utilization. HA3C operates between 43% and 67% utilization, avoiding A3C’s underuse (below 40%). This balanced profile preserves a buffer while ensuring efficient use and preventing contention at peak loads.

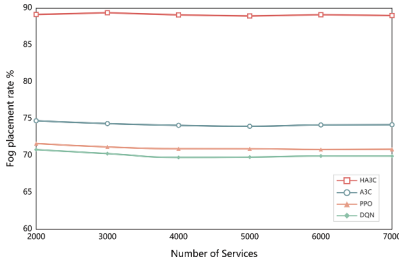


Fig. 6. Fog placement rate.

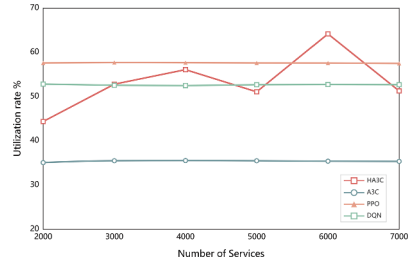


Fig. 7. Utilization rate.

To address RQ2, we conducted experiments to assess the algorithm’s performance across varying task loads. Figure 5 shows that our method sustains moderate completion times across most task scales despite increasing task volume. DQN exhibits the highest execution time, and Fig. 6 reveals its lowest service placement rate, indicating a significant migration of tasks to cloud execution, thereby increasing latency. This is juxtaposed with its highest fog node placement rate approximately 80%, suggesting that HA3C’s load balancing strategy effectively prevents fog node congestion while maintaining execution efficiency. Our algorithm underperforms compared to PPO for certain task scales. Moreover, the combined data from Fig. 6 and Fig. 7 reveal that our algorithm strikes an optimal balance between resource utilization and load distribution. Such a utilization profile preserves adequate resource headroom while ensuring effective use, thereby preventing the contention issues encountered by A3C in high load scenarios.

6 Conclusion

In the fog environment, we propose a hierarchical framework based on deep reinforcement learning, combined with the Asynchronous Advantage Actor-Critic (A3C) algorithm, to address the dynamic placement of complex services in fog environments. This framework integrates a spatiotemporal fusion architecture, a hierarchical decision-making mechanism, and a dynamic multi-objective reward function to effectively handle the placement of services with task dependencies in highly heterogeneous fog environments. It also incorporates load balancing of system nodes under limited resources. Simulation results indicate that our proposed method outperforms other traditional reinforcement learning methods. In future work, we will further explore the issue of multi-user resource competition in fog environments.

References

1. Anawar, M.R., Wang, S., Azam Zia, M., Jadoon, A.K., Akram, U., Raza, S.: Fog computing: an overview of big IoT data analytics. *Wirel. Commun. Mob. Comput.* **2018**(1), 7157192 (2018)
2. Zeng, D., Lin, G., Guo, S., Cheng, Z., Shui, Yu.: Joint optimization of task scheduling and image placement in fog computing supported software-defined embedded system. *IEEE Trans. Comput.* **65**(12), 3702–3712 (2016)
3. Sun, Y., Lin, F., Haitao, X.: Multi-objective optimization of resource scheduling in fog computing using an improved NSGA-II. *Wireless Pers. Commun.* **102**, 1369–1385 (2018)
4. Ma, X., Zhou, A., Zhang, S., Wang, S.: Cooperative service caching and workload scheduling in mobile edge computing. In: *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, pp. 2076–2085. IEEE (2020)
5. Wang, J., Hu, J., Min, G., Zomaya, A.Y., Georgalas, N.: Fast adaptive task offloading in edge computing based on meta reinforcement learning. *IEEE Trans. Parallel Distrib. Syst.* **32**(1), 242–253 (2020)
6. Chen, X., Zhang, H., Celimuge, W., Mao, S., Ji, Y., Bennis, M.: Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning. *IEEE Internet Things J.* **6**(3), 4005–4018 (2018)
7. Sharma, A., Thangaraj, V.: Intelligent service placement algorithm based on DDQN and prioritized experience replay in IoT-fog computing environment. *Internet Things* **25**, 101112 (2024)
8. Tuli, S., Ilager, S., Ramamohanarao, K., Buyya, R.: Dynamic scheduling for stochastic edge-cloud computing environments using A3C learning and residual recurrent neural networks. *IEEE Trans. Mob. Comput.* **21**(3), 940–954 (2020)
9. Kashani, M.H., Mahdipour, E.: Load balancing algorithms in fog computing. *IEEE Trans. Serv. Comput.* **16**(2), 1505–1521 (2022)
10. Yi, S., Liu, H., Chen, T., Zhang, J., Fan, Y.: A deep LSTM-CNN based on self-attention mechanism with input data reduction for short-term load forecasting. *IET Gener. Transm. Distrib.* **17**(7), 1538–1552 (2023)
11. Zou, J., Hao, T., Chen, Yu., Jin, H.: A3C-DO: a regional resource scheduling framework based on deep reinforcement learning in edge scenario. *IEEE Trans. Comput.* **70**(2), 228–239 (2020)

12. Souza, V.B.C., Ramírez, W., Masip-Bruin, X., Marín-Tordera, E., Ren, G., Tashakor, G.: Handling service allocation in combined fog-cloud scenarios. In: 2016 IEEE International Conference on Communications (ICC), pp. 1–5. IEEE (2016)
13. Trabelsi, M., Bendaoud, N.M.M., Ahmed, S.B.: Multi-objective optimization for dynamic service placement strategy for real-time applications in fog infrastructure. In: 2023 IEEE Symposium on Computers and Communications (ISCC), pp. 607–612. IEEE (2023)
14. Tavousi, F., Azizi, S., Ghaderzadeh, A.: A fuzzy approach for optimal placement of IoT applications in fog-cloud computing. *Cluster Comput.* **25**, 1–18 (2022)
15. Natesha, B.V., Guddeti, R.M.R.: Adopting elitism-based genetic algorithm for minimizing multi-objective problems of IoT service placement in fog computing environment. *J. Netw. Comput. Appl.* **178**, 102972 (2021)
16. Samani, Z.N., Mehran, N., Kimovski, D., Benedict, S., Saurabh, N., Prodan, R.: Incremental multilayer resource partitioning for application placement in dynamic fog. *IEEE Trans. Parallel Distrib. Syst.* **34**(6), 1877–1896 (2023)
17. Wang, X., Veeravalli, B., Song, J., Liu, H.: On the design and evaluation of an optimal security-and-time cognizant data placement for dynamic fog environments. *IEEE Trans. Parallel Distrib. Syst.* **34**(2), 489–500 (2022)
18. Goudarzi, M., Palaniswami, M., Buyya, R.: A distributed deep reinforcement learning technique for application placement in edge and fog computing environments. *IEEE Trans. Mob. Comput.* **22**(5), 2491–2505 (2021)
19. Sami, H., Mourad, A., Otrok, H., Bentahar, J.: Demand-driven deep reinforcement learning for scalable fog and service placement. *IEEE Trans. Serv. Comput.* **15**(5), 2671–2684 (2021)
20. Mangiaracina, G., Plebani, P., Salnitri, M., Vitali, M.: Efficient data as a service in fog computing: an adaptive multi-agent based approach. *IEEE Trans. Cloud Comput.* **11**(3), 2646–2663 (2022)
21. Yangui, S., et al.: A platform as-a-service for hybrid cloud/fog environments. In: 2016 IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN), pp. 1–7. IEEE (2016)
22. Filiposka, S., Mishev, A., Gilly, K.: Community-based allocation and migration strategies for fog computing. In: 2018 IEEE Wireless Communications and Networking Conference (WCNC), pp. 1–6. IEEE (2018)
23. Wang, L., Dai, L., Sun, L.: ConvLstm-based spatiotemporal and temporal processing models for chaotic vibration prediction of a microbeam. *Commun. Nonlinear Sci. Numer. Simul.* **140**, 108411 (2025)
24. Mahmud, R., Pallewatta, S., Goudarzi, M., Buyya, R.: iFogSim2: an extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments. *J. Syst. Softw.* **190**, 111351 (2022)